

Лабораторная работа II. Обучение языковой модели

1 Рекомендации к выполнению лабораторной работы

Для выполнения работы рекомендуется использовать язык Python, фреймворки lightning и PyTorch.

Основной код модели, обучения и вспомогательных функций должен быть реализован в .py-модулях. Jupyter-блокнот или CLI-скрипт вы можете использовать только для запуска экспериментов и визуализации промежуточных результатов. Для ЛР2 рекомендуется написать отдельный Jupyter-блокнот или CLI-скрипт, а не смешивать все в одном файле.

Эта лабораторная работа выполняется в том же репозитории, что и предыдущая работа. Для сдачи работы, создайте новую ветку из main-ветки, после чего подготовьте pull-request. Алгоритм тот же, что и в предыдущей работе.

Рекомендуемая структура проекта:

README.md	
.gitignore	
.env	- переменные окружения
requirements.txt	- фиксация окружения проекта
configs/	- конфигурационные файлы
data/	- данные
notebooks (или cli)/	- .ipynb блокноты или .py CLI-скрипты
src/	
data/	- .py файлы для обработки данных
tokenization/	- .py файлы для токенизации
models/	- .py файлы для моделей
training/	- .py файлы для кода обучения
utils.py	- утилитарные функции
checkpoints	- чекпоинты модели

В README.md опишите текущую архитектуру, причину выбора конкретной конфигурации модели и гиперпараметров обучения.

Для обучения используйте данные и токенизатор, подготовленные в ЛР1.

На GitHub загрузите чекпоинт только для финальной модели, промежуточные чекпоинты грузить не надо. Если не получается загрузить чекпоинт модели на GitHub, загрузите его на Google-диск или Яндекс-диск и в README.md добавьте на него ссылку.

2 Задание

2.1 Реализация GPT-like модели (7 баллов)

В этом задании вам необходимо реализовать все модули GPT-like модели на PyTorch, а именно:

- Модуль синусоидального позиционного кодирования;
- Модуль многоглавого маскированного внимания;
- FFN-модуль;
- Слой-трансформера. Для нормализации используйте `LayerNorm`;
- Модуль LM-head для получения логитов. Помните, что модуль функции кросс-энтропии в `torch` уже вычисляет Softmax над входами, дважды вычислять Softmax от логитов не нужно. Также учтите особенности обучения при packed batching.

При реализации слоя трансформера, используйте post-norm вариант нормализации:

$$z_1 = \text{LayerNorm}(x + \text{Attention}(x)),$$

$$z_2 = \text{LayerNorm}(z_1 + \text{FFN}(z_1)),$$

pre-norm нормализацию реализовывать пока что не нужно.

Отдельно стоит упомянуть про обучение с packed batching. Поскольку packed batching подразумевает склеивание нескольких независимых последовательностей в одну, то обрабатывать их стоит как отдельные последовательности.

Так, позиционное кодирование необходимо применять к каждой подпоследовательности отдельно: позиции токенов из одной последовательности не должны быть смещены только потому, что мы расположили эту последовательность после другой.

Также и с механизмом маскированного внимания. В этом случае обычной маски, обнуляющей все элементы матрицы внимания выше главной диагонали, недостаточно. Токены из одного объекта не должны обращать внимания на токены из другого объекта, даже если они и находятся левее. Необходимо реализовать **block-masked attention**.

Для каждого токена мы ранее хранили идентификатор s_i исходной последовательности: для паддингов идентификатор равен 0, для первого объекта равен 1, для второго 2 и так далее. Внимание между позициями i и j разрешается только в том случае, если токены принадлежат одному объекту и $j \leq i$.

$$(M)_{i,j} = (s_i = s_j) \wedge (j \leq i) \wedge (s_i \neq 0),$$

Block-Masked Attention

	A	B	C	D	E	F	G	H	<PAD>
A									
B									
C									
D									
E									
F									
G									
H									
<PAD>									

Токены	A	B	C	D	E	F	G	H	<PAD>
Индексы	1	1	1	2	2	3	3	3	0

Рис. 1: Block-Masked Attention

Кроме маски внимания, необходимо корректно считать функцию потерь. При обучении языковой модели, логиты на позиции i используются для предсказания токена на позиции $i + 1$. Если позиция i является последним токеном одной последовательности, а позиция $i + 1$ — первым токеном другой последовательности, то такой переход не должен участвовать в вычислении loss. Поэтому необходимо также реализовать маску для функции потерь:

$$M_i^{\text{loss}} = (s_i = s_{i+1}) \wedge (s_i \neq 0),$$

и считать ошибку только на тех позициях, где $M_i^{\text{loss}} = 1$.

2.2 Подготовка LightningModule (2 балла)

В предыдущем задании вы реализовали отдельные блоки модели. Теперь необходимо собрать их в единую модель и подготовить её к обучению с помощью LightningModule.

В этом задании вам необходимо:

- Реализовать класс GPT-like модели, наследуемый от LightningModule;

- Реализовать остальные необходимые методы в `LightningModule`;
- Подготовить конфигурационный файл, описывающий модель.

Объект модели и параметры ее обучения должны полностью определяться конфигурационным файлом. То есть размер словаря, число слоёв, число голов внимания, размерности слоёв, `learning rate` и другие настройки нельзя прописывать в коде модели, они должны загружаться из конфигурационного файла.

Конфиг рекомендуется прописывать в `.yaml`-файле и использовать стиль `HuggingFace`. Для загрузки конфигурационного файла рекомендуется использовать библиотеку `OmegaConf`.

Чтобы не прописывать в коде абсолютные пути, вы можете определить их в `.env`-файле и подтягивать переменные окружения. Загружать этот файл на GitHub не нужно, это считается небезопасным. Вместо этого, вы можете загрузить `.env.example`-файл или указать в `README.md` проекта его вид.

2.3 Инфраструктура обучения (7 баллов)

В этом задании вы должны реализовать инфраструктуру обучения вашей модели. Вам необходимо:

- Реализовать подсчёт метрики перплексии (`perplexity`);
- Подключить `ClearML`;
- Подключить логирование в `TensorBoard`;
- Реализовать `learning rate scheduling` и `warm-up`;
- Добавить отслеживание нормы градиентов в `ClearML`;
- Добавить обрезку градиентов (`gradient clipping`);
- Добавить сохранение чекпоинтов обучения с помощью `Callback`-класса `ModelCheckpoint`;
- Добавить восстановление модели из чекпоинта. Такая модель должна быть способна как продолжить обучение, так и генерировать текст в режиме инференса.

Во время защиты работы будьте готовы продемонстрировать все необходимые визуализации.

2.3.1 ClearML и логирование

В ClearML вы можете сохранять конфигурацию модели при запуске (полезно для анализа результатов экспериментов), а также логировать метрики с помощью `TensorBoardLogger`, например значения тренировочного лосса, валидационного лосса, значения перплексии на этапе обучения и на этапе валидации.

Эти метрики удобно отображаются в виде графиков во вкладке `Scalars`.

2.3.2 LR Scheduling и Warm-up

Для более стабильного обучения, рекомендуется настроить LR Scheduling и Warm-up. Рекомендуется реализовать в `LightningModule`-классе.

2.3.3 Норма градиентов и обрезка градиентов

Во время обучения важно отслеживать норму градиентов. Вы можете вычислять как глобальную норму для всей модели, так и локальную для каждого слоя по отдельности. Для анализа обучения модели лучше смотреть на локальные градиентные нормы.

$$\|g\| = \sqrt{\sum_i \|g_i\|^2},$$

Если норма градиентов начинает резко расти, то обучение становится нестабильным. Для стабилизации необходимо обрезать градиенты по норме (**gradient clipping**):

$$g_i \leftarrow g_i \cdot \frac{c}{\|g\|},$$

где c — максимально допустимая норма градиентов. Если же $\|g\| \leq c$, то градиенты не изменяются.

В этом задании для обрезки используйте глобальную норму. При желании, вы также можете логировать локальные нормы слоёв во время обучения.

2.4 Обучение модели (4 балла)

В этом задании вам нужно обучить реализованную модель, используя датасет и токенизатор, полученные в ЛР1. Вам необходимо:

- Обучить модель. Настройки обучения подбираете вы сами;
- Замерить значение метрики перплексии на валидационном датасете и достичь целевого порога;
- Сохранить лучший чекпоинт модели и загрузить его на GitHub (или на облачный диск);

- Вывести результат генерации модели по этому чекпоинту в режиме инференса.

В случае, если

$$\text{val_perplexity} \leq 30,$$

то за задание ставится **полный балл**. Если же

$$30 \leq \text{val_perplexity} \leq 50,$$

то за задание ставится **половина от максимального балла**. В ином случае, за это задание ставится 0 баллов.

Удачи!